short.ly

Project Documentation | Next.js 14 | Turso / libSQL | Netlify Deployed

NAVIGATION

Table of Contents

#	SECTION	DESCRIPTION
1	Project Overview	Goals, tech stack, and architecture summary
2	URL Shortener	Shorten + redirect long URLs, custom aliases, QR codes
3	Code Paste	Share syntax-highlighted code snippets with short links
4	Media Share	Upload and share images with short shareable links
5	Dashboard	Manage all links, view click analytics, delete entries
6	QR Code Generation	Auto-generate scannable QR codes for every item
7	Dark / Light Mode	System-aware theme toggle
8	Database Schema	Turso (cloud SQLite) tables and indexes
9	API Reference	All REST endpoints and their behaviour
10	Deployment	Netlify + Turso configuration

SECTION 1

Project Overview

short.ly is a full-stack web application built with **Next.js 14** that combines three powerful sharing utilities in one place: a URL shortener, a code paste tool, and an image sharing service. Every shared item gets a unique 6-character short code (or a custom alias) and an auto-generated QR code.

> **Deployment:** The application is hosted on **Netlify** using serverless API routes (Next.js API handlers compiled to Netlify Functions). Data is persisted in **Turso**, a cloud-hosted libSQL (SQLite-compatible) database, and file blobs are stored in **Netlify Blobs**.

Technology Stack

Next.js 14 (App Router)

React framework — Server Components, Client Components, API Routes and file-based routing

Turso / libSQL

Cloud SQLite database. Accessed via the `@libsql/client` package from server-side API routes

Tailwind CSS

Utility-first CSS framework for responsive, themed UI components

Netlify Blobs

Object storage used to persist uploaded images, keyed by the media item's short code

TypeScript

End-to-end type safety for API payloads, database rows, and React props

qrcode

Node.js QR code library used server-side to generate PNG QR codes as base64 data URIs

Application Pages

ROUTE	PAGE	PURPOSE
`/`	Home	URL Shortener form and hero landing section
`/paste`	Paste Code	Code snippet submission form
`/media`	Media Share	Image upload and sharing form
`/dashboard`	Dashboard	Tabbed view of all links, pastes, and media items
`/[shortCode]`	Redirect	Resolves a short code and redirects to the original URL
`/paste/[shortCode]`	Paste Viewer	Displays syntax-highlighted code with copy button
`/media/[shortCode]`	Media Viewer	Displays the shared image with download link

SECTION 2

URL Shortener

The core feature of short.ly lets users paste any long URL and receive a compact short link. The short link automatically redirects visitors to the original destination and tracks how many times it has been visited.

How to Use

Enter a long URL
Paste any valid URL into the main input field on the home page.

Optional: Set a custom alias
Type a memorable word (e.g. "my-blog") to use as the short code instead of the random one. Only alphanumeric characters, hyphens, and underscores are accepted.

Optional: Set an expiry date
Pick a date after which the short link will stop redirecting visitors.

Click "Shorten URL"
The form POSTs to `/api/shorten`. The result card appears below the form with the short link and QR code.

Copy or scan the result
Use the copy button to copy the short URL to clipboard, or scan the QR code with any mobile device.

Key Features

FEATURE	DESCRIPTION	IMPLEMENTATION DETAIL
Short Code Generation	Auto-generates a random 6-character alphanumeric code	`generateShortCode()` in `lib/api.ts` — iterates 6 times picking chars from `a-z0-9` using `Math.random()`
Custom Alias	User-defined short code instead of the random one	Uniqueness is verified against the `links` table before insert. Input is sanitised in the React form to allow only `[a-zA-Z0-9-_]`
Link Expiry	Links can expire on a chosen date	`expires_at` is stored as ISO 8601 string. On resolution, the API checks `new Date(expiresAt) < new Date()` and blocks the redirect if expired
Click Tracking	Every redirect increments the visit counter	`UPDATE links SET clicks = clicks + 1` runs atomically on the Turso row when a valid (non-expired) short code is resolved
QR Code	Scannable QR code auto-generated for each link	Fetched via `/api/qr?url=...` which calls the `qrcode` npm library server-side and returns a base64 PNG data URI

Implementation Snippet — Short Code Resolution

```
// lib/api.ts — resolveShortCode() const { rows } = await db.execute({ sql: 'SELECT * FROM links WHERE short_code = ?', args: [shortCode], }) const isExpired = rowNum(row, 'is_expired') === 1 || (expiresAt ? new Date(expiresAt) < new Date() : false) // Only count the click if link is still active if (!isExpired) { await db.execute({ sql: 'UPDATE links SET clicks = clicks + 1 WHERE short_code = ?', args: [shortCode], }) }
```

SECTION 3

Code Paste

Inspired by tools like Pastebin and paste.ee, the Code Paste feature allows developers to share code snippets with full syntax highlighting, accessible via a short link. Navigate to **/paste** to use it.

Supported Languages (30+)

The language selector supports the following modes for syntax highlighting:

JavaScript · TypeScript · Python · Java · C / C++ / C# · Go · Rust · PHP · Ruby · Swift / Kotlin · HTML / CSS / SCSS · SQL · Bash / PowerShell · JSON / YAML / XML · Markdown · Dockerfile · GraphQL · R / Lua / Dart / Scala · Plain Text

How to Use

1. **Go to /paste**
   Click "Paste Code" in the top navigation bar.
2. **Set an optional title**
   Give the paste a descriptive name shown in the viewer header.
3. **Select the programming language**
   Choose from 30 languages in the dropdown — this controls syntax highlighting in the viewer.
4. **Paste your code**
   The monospace `<textarea>` is resizable and spell-check is disabled.
5. **Optionally set alias and expiry, then click "Create Paste"**
   POSTs to `/api/paste`. Returns a short URL (e.g. `short.ly/paste/ab12cd`) and QR code.
6. **Share the link**
   Anyone visiting the URL sees the rendered, read-only code viewer with a one-click copy button.

Implementation Details

ASPECT	DETAIL
Storage	Code is stored as a plain `TEXT` column in the `code_pastes` Turso table — no file system involved
Syntax Highlighting	The paste viewer uses the `highlight.js` library, applied client-side via a `useEffect` hook that calls `hljs.highlightElement()` on the `<pre><code>` block after mount
Alias Uniqueness	Custom aliases are checked across **both** `links` and `code_pastes` tables simultaneously using `Promise.all()`
Click Tracking	Same mechanism as URL links — increments `clicks` on first load of the paste viewer page
Expiry	Expired pastes show an "Expired" banner in the viewer instead of redirecting

SECTION 4

Media Share

The Media Share feature lets users upload images and receive a short link that displays a full-page image viewer. Files are stored in **Netlify Blobs** (object storage) keyed by the short code, while metadata is stored in Turso.

Supported Formats & Limits

PROPERTY	VALUE
Accepted MIME types	JPEG, PNG, GIF, WebP, SVG, AVIF
Maximum file size	5 MB (enforced client-side and server-side)
Upload method	`multipart/form-data` via the Fetch API using `FormData`

How to Use

1. **Navigate to /media**
   Click "Media Share" in the top navigation bar.
2. **Drop or browse for an image**
   The drag-and-drop zone accepts any supported image type. A real-time preview is shown immediately after selection, along with the file name and size.
3. **Optional: add a title, custom alias, or expiry date**
4. **Click "Share Image"**
   The file is uploaded as a `FormData` POST to `/api/media`. The server stores the blob in Netlify Blobs and the metadata in Turso.
5. **Share the returned link or QR code**
   Recipients see a full-screen responsive image viewer at `/media/[shortCode]` with a download button.

Implementation Details

ASPECT	DETAIL
Drag & Drop	Implemented via HTML5 `onDragOver` / `onDrop` events on a styled `<div>`. A hidden `<input type="file">` is triggered on click via a `useRef` handle
Client Preview	`URL.createObjectURL(file)` creates a temporary local URL for the `<img>` preview without uploading
Blob Storage	Server-side API route uses the `@netlify/blobs` SDK — `store.set(shortCode, buffer)` — to persist the raw file bytes
Raw Download	`GET /api/media/[shortCode]/raw` fetches the blob from Netlify Blobs and streams it back with the correct `Content-Type` header
Metadata	Filename, MIME type, size in bytes, short code, title, clicks, timestamps stored in `media_shares` Turso table

SECTION 5

Dashboard

The Dashboard (**/dashboard**) provides a central management interface for all items created in the application. It is split into three tabs: *Links*, *Pastes*, and *Media*.

Dashboard Features

Click Analytics

Each row shows the total click/view count for that item in real time.

One-Click Copy

A copy button next to every short URL instantly copies it to the clipboard.

Delete

Every row has a delete button that permanently removes the item from the database.

QR Modal

Clicking the QR icon opens a full-resolution QR code in a modal overlay, ready to scan or download.

Expiry Status

Expired items are visually dimmed and show an "Expired" badge so you can identify and clean them up.

Sortable Table

All items are listed newest-first with short code, original URL, clicks, created date, and expiry columns.

Implementation Details

ASPECT	DETAIL
Data Fetching	The dashboard page is a Server Component that fetches all three lists from Turso via `getLinks()`, `getPastes()`, and `getMediaShares()` in parallel using `Promise.all()` before rendering
Tab State	A lightweight client-side `useState` controls which tab is shown — no router changes needed, tabs are purely presentational
Delete Action	Clicking delete calls `DELETE /api/links/:id`, `DELETE /api/paste/:id`, or `DELETE /api/media/:id`. On success the row is removed from local React state without a full page refresh
Copy Button	`navigator.clipboard.writeText()` copies the URL; the button icon switches from Clipboard to Checkmark for 2 seconds as visual feedback

SECTION 6

QR Code Generation

Every item (short URL, paste, or media) automatically gets a QR code. QR codes are generated **server-side** to keep the bundle lean and avoid compatibility issues in serverless environments.

Flow

Result card or modal is rendered
The React component calls `/api/qr?url=<encoded-url>` via `fetch`.

Server generates the QR PNG
The API route imports `qrcode` and calls `QRCode.toDataURL(url, { ...options })` which returns a base64-encoded PNG data URI.

Data URI is rendered in an <img> tag
No file is written to disk — the base64 string is returned as JSON and placed directly into `src`.

Modal allows full-size download
The `QRCodeModal` component includes a download anchor that triggers a browser save of the PNG.

QR API Endpoint

```
GET /api/qr?url={encodedUrl}
```

Returns `{"qr": "..."}`

> **Why server-side?** The `qrcode` npm package uses Node.js Canvas internally. Running it in the browser or in certain edge runtimes causes errors, so all generation happens in the Next.js API route (a full Node.js serverless function on Netlify).

SECTION 7

Dark / Light Mode

short.ly ships with a full dark-mode theme that respects the user's system preference and allows manual override via a toggle button in the navigation bar.

Implementation

ASPECT	DETAIL
Library	`next-themes` wraps the root layout in a `ThemeProvider` that handles SSR hydration and persists the choice to `localStorage`
Toggle Button	`ThemeToggle` component uses `useTheme()` to read and set the current theme, switching between a Sun icon (light) and Moon icon (dark)
CSS Variables	All colours are defined as CSS custom properties. Tailwind's dark-mode (`class` strategy) applies a `.dark` class to `<html>` and the variables flip automatically
Default	`attribute="class"` and `defaultTheme="system"` — the app defaults to whatever the OS prefers on first visit

SECTION 8

Database Schema

All data is stored in **Turso**, a distributed libSQL (SQLite-compatible) database. The schema is auto-initialised on first API call via `initDb()` in `lib/db.ts` using `CREATE TABLE IF NOT EXISTS` DDL, making the app self-bootstrapping.

TABLE: LINKS

COLUMN	TYPE	DESCRIPTION
`id`	TEXT PK	Unix timestamp string used as unique ID
`original_url`	TEXT NOT NULL	The full long URL to redirect to
`short_code`	TEXT UNIQUE	6-char random or user-defined alias
`clicks`	INTEGER	Number of times the short link was accessed (default 0)
`created_at`	TEXT	ISO 8601 creation timestamp
`expires_at`	TEXT (nullable)	ISO 8601 expiry timestamp — `NULL` means never expires
`is_expired`	INTEGER	Manual override flag (0 = active, 1 = expired)

TABLE: CODE_PASTES

COLUMN	TYPE	DESCRIPTION
`id`	TEXT PK	Unix timestamp string
`code`	TEXT NOT NULL	Full code snippet text
`language`	TEXT	Language identifier for highlight.js (default: `plaintext`)
`short_code`	TEXT UNIQUE	Short identifier for the paste
`title`	TEXT (nullable)	Optional display title
`clicks`	INTEGER	View count
`created_at`	TEXT	ISO 8601 creation timestamp
`expires_at`	TEXT (nullable)	Optional expiry timestamp
`is_expired`	INTEGER	Manual override flag

TABLE: MEDIA_SHARES

COLUMN	TYPE	DESCRIPTION
`id`	TEXT PK	Unix timestamp string
`filename`	TEXT NOT NULL	Original file name of the uploaded image
`mime_type`	TEXT NOT NULL	MIME type, e.g. `image/png`
`size_bytes`	INTEGER	File size in bytes
`short_code`	TEXT UNIQUE	Short identifier (also the Netlify Blob key)
`title`	TEXT (nullable)	Optional display title
`clicks`	INTEGER	View count
`created_at`	TEXT	ISO 8601 creation timestamp
`expires_at`	TEXT (nullable)	Optional expiry timestamp
`is_expired`	INTEGER	Manual override flag

> **Indexes:** Each table has a dedicated index on `short_code` (`CREATE INDEX IF NOT EXISTS idx_*_short_code`) for O(log n) lookups on every redirect and resolve operation.

Helper Utilities (lib/db.ts)

FUNCTION	PURPOSE
`getDb()`	Singleton factory — creates the libSQL client once per serverless instance using env vars `TURSO_DATABASE_URL` and `TURSO_AUTH_TOKEN`
`initDb()`	Runs all DDL migrations idempotently and seeds sample link data if the `links` table is empty
`rowStr(row, col)`	Safely casts a libSQL Row column to `string` (returns `''` for nulls)
`rowStrNull(row, col)`	Same but returns `null` instead of empty string
`rowNum(row, col)`	Safely casts to `number` (returns `0` for nulls)

SECTION 9

API Reference

All API routes live under `app/api/` and are executed as Netlify Functions (full Node.js runtime). Every endpoint calls `initDb()` to ensure the schema exists before running any SQL.

URL Shortener Endpoints

`POST` `/api/shorten`

Body: `{"url": "...", "customAlias"?: "...", "expiresAt"?: "YYYY-MM-DD"}`
Validates the URL, checks alias uniqueness, generates a short code, inserts into `links`, returns the full `ShortenedLink` object.

`GET` `/api/links`

Returns all links ordered by `created_at DESC` for the dashboard.

`DELETE` `/api/links/[id]`

Permanently deletes a link row by its ID.

Code Paste Endpoints

`POST` `/api/paste`

Body: `{"code": "...", "language": "...", "title"?: "...", "customAlias"?: "...", "expiresAt"?: "..."}`
Validates code is non-empty, checks alias across both `links` and `code_pastes` tables, inserts and returns the paste.

`GET` `/api/paste/[shortCode]`

Resolves a paste short code, increments click counter, returns full paste data.

`DELETE` `/api/paste/[id]`

Permanently deletes a paste by ID.

Media Share Endpoints

```
POST /api/media
```

Body: `multipart/form-data` with fields `file`, optional `title`, `customAlias`, `expiresAt`.

Validates file type and size, stores blob in Netlify Blobs, inserts metadata into `media_shares`.

```
GET /api/media/[shortCode]/raw
```

Fetches the raw image blob from Netlify Blobs and streams it back with correct MIME type headers.

```
DELETE /api/media/[id]
```

Deletes the media metadata from Turso and the blob from Netlify Blobs.

Utility Endpoints

```
GET /api/qr?url={encodedUrl}
```

Generates a QR code PNG server-side and returns it as a base64 data URI JSON response.

SECTION 10

Deployment

The project is deployed to **Netlify** using Next.js's Netlify adapter. Database reads and writes go to **Turso** (cloud SQLite), and binary file data is stored in **Netlify Blobs**.

Required Environment Variables

VARIABLE	WHERE TO SET	PURPOSE
TURSO_DATABASE_URL	Netlify Dashboard → Site → Environment Variables	libsql:// connection URL for the Turso database
TURSO_AUTH_TOKEN	Netlify Dashboard → Site → Environment Variables	Authentication token for the Turso database

netlify.toml Configuration

```
# netlify.toml [build] command = "npm run build" publish = ".next" [[plugins]] package = "@netlify/plugin-nextjs"
```

Local Development

```
# 1. Clone and install dependencies npm install # 2. Create .env.local with your Turso credentials TURSO_DATABASE_URL=libsql://<db>.turso.io TURSO_AUTH_TOKEN=<your-token> # 3. Start the development server npm run dev # starts at http://localhost:3000
```

Deployment Checklist

STEP	ACTION	STATUS INDICATOR
1	Push code to GitHub repository	Git push
2	Connect GitHub repo to Netlify	Netlify dashboard
3	Set `TURSO_DATABASE_URL` and `TURSO_AUTH_TOKEN` in Netlify	Environment vars section
4	Trigger deploy (auto on push or manual)	Netlify build logs
5	Visit the deployed URL and test all three features	Green status in Netlify

short.ly — Built with Next.js 14 · Turso · Netlify

Documentation generated March 2026 · subtach/shortly.github.io